



POLITECNICO
MILANO 1863

BEST BIKE PATHS

ITD

Filippo Raimondi

Matteo Sala

Deliverable:	ITD
Title:	Implementation and Test Derivable
Authors:	Raimondi Filippo, Sala Matteo
Version:	1.0
Date:	01-02-2026
Download page:	https://github.com/m-tteo/sw-eng-2-raimondi-sala
Copyright:	Copyright © 2026, Raimondi Filippo, Sala Matteo – All rights reserved

Contents

Table of Contents	2
1 Introduction	4
1.1 Scope of the document	4
1.2 Definitions, Acronyms, and Abbreviations	4
1.3 Revision History	4
1.4 Reference Documents	5
1.5 Document Structure	5
2 Product Functions	6
2.1 Implemented functions	6
2.2 Discarded functions and details	6
2.3 Other changes	6
3 System Development Frameworks	7
3.1 Adopted frameworks	7
3.2 Adopted programming languages	7
3.3 Adopted middlewares and libraries	7
3.4 Adopted APIs	8
4 Source Code Structure	9
4.1 Backend	9
4.2 Frontend	10
5 Testing Activity	11
5.1 Test Cases	11
5.1.1 User Management Services	11
5.1.2 Bike Path Operations	11
5.1.3 Trip Service	12
5.1.4 Obstacle Management	12
5.1.5 External Integrations	12
6 Installation Guide	13
7 Prerequisites	13
8 Backend Setup	13
8.1 Build from Source	13
8.2 Download Pre-compiled JAR	13
8.3 Run the Backend	13
9 Frontend Setup	13
9.1 Build from Source	13
9.2 Download Pre-compiled Executable	14
9.3 Run the Frontend	14
9.4 Access the Application	14
10 Configuration Options	14
10.1 Backend Parameters	14
10.2 Frontend Parameters	14

11 Use of Generative AI Tools	15
12 Effort Spent	16
References	17

1 Introduction

1.1 Scope of the document

The purpose of this document is to present the implemented prototype of the project. Therefore, this document presents the functions of the software, the frameworks that have been used to implement them, how they have been tested, and finally, a guide on how to run the project.

1.2 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface
BBP	Best Bike Paths
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
ITD	Implementation and Test Deliverable
JPA	Java Persistence API
JSON	JavaScript Object Notation
JTS	Java Topology Suite
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single Page Application
UI	User Interface
URL	Uniform Resource Locator

Table 1: Definitions, Acronyms, and Abbreviations

1.3 Revision History

Version	Date	Authors
1.0	1-02-2026	Raimondi F., Sala M.

Table 2: Document Revision History

1.4 Reference Documents

1. A.Y. 2025-2026 Software Engineering 2 - Project Specification [2] [1].

1.5 Document Structure

The document is organized into the following sections:

Introduction: The first section of this document, containing the Scope of the document, the definitions and acronyms, revision history and reference documents.

Product Functions: This section describes the implemented functions, lists the discarded functions that were excluded from the final implementation, and details other structural changes.

System Development Frameworks: This section lists the technologies used to develop the application, including adopted frameworks, programming languages, middlewares, and external APIs.

Source Code Structure: This section explains the organization of the source code directories.

Testing Activity: This section details the testing strategy.

Installation Guide: This section provides step-by-step instructions to build and run both the backend and frontend components of the system.

Use of Generative AI Tools: This section documents the use of AI assistance during development and documentation.

Effort Spent: This final section presents a breakdown of the hours spent by each group member for writing this document.

2 Product Functions

2.1 Implemented functions

F1. User Management

This function is fundamental as it allows a user to register and gain access to the system. After submitting the registration details or logging in with existing credentials, the system will provide the user with a JSON Web Token. Finally, the user is able to use this token to authenticate all subsequent requests to the application APIs, manage their profile information, and log out.

F2. Manual Trip Recording

This function allows a cyclist to insert a trip into the system without having to track it in real-time. After selecting the origin, destination, and timing, the system will calculate the route details using the Mapbox integration and enrich the record with historical weather data from Open-Meteo. Finally, the cyclist is able to view this trip in their personal history immediately after creation.

F3. Manual Bike Path Creation

This function allows a user to define a new bike path in the system. The system will provide the user with the calculated route and validate any reported obstacle, ensuring it lies within a specific distance from the path geometry. Finally, the user is able to update the path status (visibility and maintenance status) or delete it if the user is the creator.

F4. Bike Path Search

This function allows a user to discover bike paths created by the community. The system will provide the user with a list of published bike paths that match the specified search radius around a starting point and a destination. This data should allow the user to choose the best rated path for their commute.

F5. Collaborative Maintenance

This function allows users to report and modify obstacles on public bike paths, changing their positions, removing them, or editing their severity.

2.2 Discarded functions and details

This is a list of functions or functional details that are described in the previous documents but have been excluded from the implementation of the system.

- **F6. Automatic Trip Recording:** this functionality was not implemented, it required a continuous stream of GPS data from the client, which was not required in the implementation project.
- **F7. Automatic Bike Path Creation:** this feature, which intended to automatically create bike paths through GPS tracking and sensor data with obstacle detection and user validation, was not implemented since it was not required in the implementation project.

2.3 Other changes

For the prototype implementation, the database technology was changed from PostgreSQL with PostGIS to H2 in-memory database. Consequently, spatial operations are handled in the backend using the Java Topology Suite (JTS) library instead of PostGIS database extensions. These changes simplify the infrastructure setup for prototyping and testing purposes.

3 System Development Frameworks

3.1 Adopted frameworks

The development of the application has been facilitated by the adoption of notable frameworks.

- **Spring Boot:** we used Spring Boot as the backend framework, simplifying the development of the application. It provides the embedded server and auto-configuration required to run the Java-based system.
- **Vue.js:** for the frontend, we decided to use Vue.js, which enabled fast development of an interactive UI.
- **JUnit and Mockito:** for testing the backend we used JUnit and Mockito to perform unit testing with mocked dependencies.

3.2 Adopted programming languages

Java The backend has been completely written in Java, which has been used to define the entities, the business logic services, and the REST controllers that expose the API to the client.

Java Pros

- **Statically Typed:** being strongly typed ensures that type errors are caught at compile-time rather than runtime, making the system more robust.
- **Rich Ecosystem:** the availability of libraries like JTS for geospatial operations sped up development.
- **Multithreading:** Java's native support for multithreading allows the application to handle concurrent tasks, such as fetching weather data asynchronously.

Java Cons

- **Memory Consumption:** the Java Virtual Machine tends to consume more memory compared to lighter runtime environments.

TypeScript The frontend has been written in TypeScript.

TypeScript Pros

- **Type Safety:** it brings the advantages of static typing to JavaScript.
- **Maintainability:** in complex frontend projects, explicit interfaces and types make the code easier to read and maintain.

Other languages Additionally, HTML and CSS were used to define the structure and styling of the pages.

3.3 Adopted middlewares and libraries

Backend

Middlewares

- **Spring Security:** used for managing authentication and authorization.
- **Spring Data JPA:** used for database abstraction, allowing data manipulation via Repository interfaces.

Libraries

- **JWT**: used for JWT token creation and validation.
- **MapStruct**: used for compile-time Entity-DTO mapping.
- **Lombok**: used to reduce boilerplate code.
- **JTS (Java Topology Suite)**: used for geospatial operations and geometric calculations.
- **Resilience4j**: used for rate limiting requests to external APIs.
- **Caffeine**: used as high-performance caching library for external API responses.
- **H2 Database**: used as in-memory database for development and testing.

Frontend

Libraries

- **Pinia**: used as the state manager.
- **Vue Router**: used for client-side navigation, mapping URLs to Vue components.
- **Axios**: used to perform asynchronous HTTP requests to the backend.
- **Mapbox GL**: used for interactive map rendering and visualization.
- **Tailwind CSS**: used to rapidly style components, ensuring visual consistency.
- **DaisyUI**: used as UI component library built on Tailwind CSS.
- **Lucide Vue Next**: used for icons.

3.4 Adopted APIs

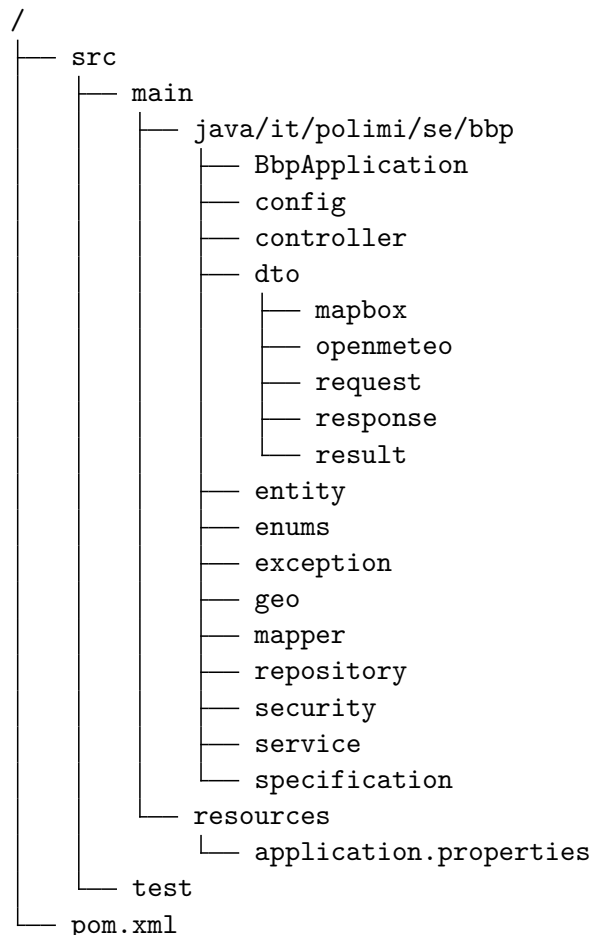
The system relies on the following external APIs:

- **Mapbox**: used for geocoding (address to coordinates conversion and reverse), and cycling route calculation.
- **Open-Meteo**: used to retrieve historical weather data for trip enrichment.

4 Source Code Structure

The source code is organized in two main directories, separating the frontend logic from the backend logic.

4.1 Backend



config Contains Spring configuration classes, and external services such as Mapbox and OpenMeteo.

controller Contains REST Controllers that expose API endpoints, handle HTTP requests, and delegate logic to services.

dto Collects Data Transfer Objects, separated into request and response, as well as DTOs specific to external APIs.

entity Defines JPA entities that map database tables.

enums Enumeration types for domain values.

exception Centralized exception handling and custom exception definitions.

geo Utility classes for geospatial calculations.

mapper Interfaces for automatic conversion between Entities and DTOs.

repository Interfaces for database interaction.

security Components for JWT authentication.

service Core business logic of the application.

specification Classes for constructing dynamic queries used for search filters.

4.2 Frontend

```

/
├── public
├── src
│   ├── api
│   ├── components
│   ├── composables
│   ├── config
│   ├── constants
│   ├── router
│   ├── services
│   ├── stores
│   ├── styles
│   ├── types
│   ├── utils
│   ├── views
│   ├── App.vue
│   └── main.ts
├── index.html
├── package.json
├── tailwind.config.ts
└── vite.config.ts

```

api HTTP client configuration and request interceptor management.

components Reusable Vue components in the interface.

composables Functions encapsulating reusable stateful logic.

config Configuration files for environment variables and external services.

constants Application-wide constant values and configuration parameters.

router Client-side routing configuration defining page navigation.

services Modules interfacing the frontend with backend APIs.

stores Global application state management via Pinia.

styles Global CSS files.

types Shared TypeScript interface definitions to ensure strong data typing.

views Main application pages that assemble the various components.

utils Pure utility functions for validation, date formatting, and static calculations.

5 Testing Activity

5.1 Test Cases

Unit tests were developed using the JUnit framework and Mockito to isolate business logic and simulate external dependencies.

5.1.1 User Management Services

Testing for user management covers both authentication and profile modification. The following tests were executed:

- **Authentication:**
 - Registration with valid information.
 - Registration with an existing username.
 - Registration with an email already in use.
 - Login with valid credentials.
 - Login with non-existent email.
 - Login with incorrect password.
- **Profile Management:**
 - Full user profile update.
 - Partial update.
 - Update with a username already taken by another user.
 - Update with an email already taken by another user.
 - Update without any variation.
 - Deletion of the current user account.

5.1.2 Bike Path Operations

Tests related to bike paths verify the complete lifecycle of a path, from manual creation to search and privacy management. The following tests were executed:

- **CRUD Operations:**
 - Manual creation of a path.
 - Retrieval of a public path.
 - Retrieval of a private path by the creator.
 - Attempted retrieval of a private path by a third party.
 - Retrieval of a non-existent path.
 - Update of path metadata by the owner.
 - Concurrent update with an obsolete version.
 - Deletion of a path by the owner.
- **Search:**
 - Path search with filters and data enrichment.
 - Geographic search within a specific radius between origin and destination.
 - Result filtering based on real distance.
 - Pagination handling with valid and invalid parameters.

5.1.3 Trip Service

Tests related to the recording and management of personal trips, verifying integration with weather services. The following tests were executed:

- **Trip Recording:**
 - Manual recording of a trip with valid data.
 - Trip recording even in case of external weather service failure.
- **Trip Management:**
 - View trip details by the owner.
 - Attempt to view another user's trip.
 - Deletion of an existing trip.
 - Search and pagination of the user's trip history.

5.1.4 Obstacle Management

Tests to verify the spatial logic for placing obstacles along the path and calculating the safety score. The following tests were executed:

- **Obstacle Lifecycle:**
 - Creation and saving of obstacles within the path buffer.
 - Rejection of obstacle creation if placed too far from the path.
 - Update of an existing obstacle.
 - Addition of new obstacles during an update phase.
- **Scoring:**
 - Score calculation with an empty obstacle list.
 - Penalty calculation based on severity and active/inactive status of obstacles.

5.1.5 External Integrations

Tests were performed to ensure system resilience against failures from third-party APIs:

- **Mapbox Service:**
 - Geocoding of valid addresses.
 - Handling of invalid or not found addresses.
 - Handling of network errors.
 - Handling of API limits.
 - Bike route calculation with multiple waypoints.
- **Open-Meteo Service:**
 - Retrieval of correct historical weather data.
 - Handling of requests with dates that are too old.
 - Handling of timeouts and server errors.
 - Handling of empty or malformed responses.

6 Installation Guide

7 Prerequisites

Before proceeding, ensure the following tools are installed on your machine:

- **Java Development Kit:** Version 25 or higher
- **Bun:** Visit <https://bun.sh> for installation
- **Mapbox API Key:** Visit <https://www.mapbox.com/> to obtain a free API key (default token settings are sufficient)

8 Backend Setup

8.1 Build from Source

Navigate to the backend directory:

```
cd implementation/backend/bbp
```

Build the executable JAR file:

```
./mvnw clean package -DskipTests
```

Note: The `-DskipTests` flag skips unit tests. Remove it to run tests during build.

8.2 Download Pre-compiled JAR

Alternatively, download the pre-compiled JAR file from the [releases page](#).

8.3 Run the Backend

Execute the application with your Mapbox API key:

```
java -jar bbp-backend.jar --mapbox.api.key=YOUR_MAPBOX_KEY
```

9 Frontend Setup

9.1 Build from Source

Navigate to the frontend directory:

```
cd implementation/frontend/bbp
```

Install dependencies:

```
bun install
```

Build the production executable:

```
bun run build:prod
```

9.2 Download Pre-compiled Executable

Alternatively, download the appropriate pre-compiled executable for your platform from the [releases page](#).
For macOS/Linux only: Navigate to the directory containing the executable and make it executable:

```
chmod +x bbp-frontend-<platform>
```

Replace <platform> with your specific file suffix (e.g., linux-x64). Windows users can skip this step.

9.3 Run the Frontend

Execute the frontend with your Mapbox API key:

```
./bbp-frontend-<platform> --mapbox.api.key=YOUR_MAPBOX_KEY
```

Replace <platform> with your specific file suffix (e.g., linux-x64, macos-arm64).

9.4 Access the Application

Open your browser and navigate to:

<http://localhost:3000>

Note: If you specified a custom port with `--frontend.port`, use that port instead of 3000.

10 Configuration Options

10.1 Backend Parameters

- `--mapbox.api.key`: Your Mapbox API key (required)
- `--server.port`: Backend port (default: 8080)
- `--client.port`: Frontend port for CORS configuration (default: 3000, must match the frontend's actual port)

10.2 Frontend Parameters

- `--mapbox.api.key`: Your Mapbox API key (required)
- `--frontend.port`: Frontend port (default: 3000)
- `--backend.port`: Backend port to connect to (default: 8080, must match the backend's actual port)

11 Use of Generative AI Tools

Generative AI tools were used during the development of this project to support both documentation and implementation activities.

For the writing of this document, AI assistance was employed to improve clarity, ensure consistency in terminology, and refine the structure of technical descriptions.

During the implementation phase, AI tools were utilized as coding assistants to accelerate development, suggest code patterns, debug issues, and provide guidance on framework-specific best practices. All AI-generated code suggestions were reviewed, tested, and adapted to meet the project's specific requirements and coding standards.

12 Effort Spent

This section includes the table of hours worked by each group member to complete this document.

Member of group	Effort spent
Raimondi Filippo	2 <i>h</i>
Sala Matteo	2 <i>h</i>
Total	4 h

Table 3: Effort Spent on ITD

References

- [1] A.Y. 2025-2026 Software Engineering 2 - I&T assignment goal, schedule, and rules.
- [2] A.Y. 2025-2026 Software Engineering 2 - Requirement Engineering and Design Project: goal, schedule, and rules.